LangSmith vs Langfuse

Same agent. Same four test questions. Same `gpt-4o` judge. The only thing that changes is which tool is watching.

Every screenshot below is from this lab. Every number is measured, not quoted from a docs page.

- **LangSmith**: hosted, at smith.langchain.com
- **Langfuse**: self-hosted, six Docker containers on localhost:3000, `v4.6.0 OSS`

What we are watching

A small LangGraph agent with three tools:

- `calculator`, always works
- `get_weather`, fake data that **fails 20% of the time on purpose**
- `search_docs`, tiny RAG over five markdown files

The 20% failure is the interesting part. It is what makes the error screens below real instead of staged.

Contents

Document	Covers
01-tracing.md	Turning tracing on, reading a trace, error handling, grouping runs
02-evaluation.md	Golden datasets, LLM-as-judge, measured scores
03-features.md	Dashboards, and what each tool has that the other does not
04-cost.md	Pricing for both, and measured CPU/RAM/disk for self-hosted Langfuse
05-scale.md	How many traces self-hosted Langfuse handles, load tested, versus LangSmith's limits

PDF versions of all of the above are in pdf/, including a single combined langfuse-vs-langsmith-FULL.pdf with everything in one file. Screenshots are embedded, so the PDFs are self-contained and safe to send on.

Running this yourself

```
# plain app, traced by LangSmith automatically
uv run python -m src.agent "what's the weather in Chennai and what's 100/7?"

# same agent, explicitly traced into Langfuse
uv run python -m src.obs_langfuse.traced_run "what's the weather in Chennai and what's 100/7?"

# datasets (run once each)
uv run python -m src.obs_langsmith.dataset
uv run python -m src.obs_langfuse.dataset

# evaluations
uv run python -m src.obs_langsmith.evaluate
uv run python -m src.obs_langfuse.evaluate
```

To force a failure instead of waiting for the 20% dice:

```
PYTHONPATH=. uv run python -c "
import random
random.random = lambda: 0.0
from dotenv import load_dotenv; load_dotenv()
from src.agent.graph import graph
graph.invoke({'messages': [('user', 'what is the weather in Chennai?')]}, config={'run_name': 'for
"
```

Verdict

What you care about	Pick
Tracing working in ten seconds	**LangSmith**
Data must not leave your network	**Langfuse**
Seeing *where* an agent broke	**Langfuse**
Judge explaining its scores	**Langfuse**
Nothing to operate	**LangSmith**
Filtering on your own metadata keys	**LangSmith**

There is no overall winner. If your data can leave the building, LangSmith gets you further in an afternoon and there is nothing to maintain. If it cannot, Langfuse is a real replacement rather than a downgrade, and its error views are genuinely better. The price is six containers and a port conflict.